

九齿 skill 创新挑战_T3-1-1_赛题报告

参赛赛题：T3-1-1 NineToothed 算子开发 Skill

提交 skill：nine-toothed-operator-dev

小组名称：tusu-code（个人参赛）

成员：tusu-code（GitHub: tusu-code）

日期：2026-07-02

1. 摘要

本提交构建了一个面向 NineToothed 算子开发任务的可复用 .skill 包，目标是让 AI 智能体在接到算子开发、测试、性能分析、AOT 构建或失败诊断任务时，能够按照固定闭环完成工作：先抽取算子语义、shape、dtype、broadcast、layout 和边界条件，再阅读仓库相似实现，设计 arrangement，编写最小补丁，补充 correctness test，运行验证命令，并在性能敏感或失败场景中记录 benchmark、generated source、AOT 或失败诊断证据。

本 skill 的重点不是堆叠长文档，而是把 NineToothed 特有的开发动作转化为 AI 必须执行的检查清单，尤其包括 arranged tensor 外层 shape 对齐、per-program block 语义、Tensor(..., other=identity) 边界填充值、ninetoothed.debugging.simulate_arrangement 调试、PyTorch 参考对齐、benchmark 记录和最小修复闭环。

2. 包结构

```
nine-toothed-operator-dev/  
  SKILL.md  
  README.md  
  HONOR_CODE.md  
  REFERENCE.md  
  PR_DESCRIPTION_TEMPLATE.md  
  agents/  
    openai.yaml  
  references/  
    nine-toothed-api-notes.md  
    operator-patterns.md  
    testing.md  
    performance.md  
    failure-diagnosis.md  
    final-submission.md  
    repository-pattern-index-ninetoothed.md  
    repository-pattern-index-examples.md  
  scripts/  
    scan_repo.py  
    build_pattern_index.py  
    make_selftest_task.py  
    check_submission.py  
  examples/  
    01-elementwise-add/task.md  
    02-softmax-reduction/task.md  
    03-layout-stride-offset/task.md  
    04-performance-diagnosis/task.md  
    05-from-scratch-gelu/task.md  
    06-from-scratch-l2norm/task.md  
    07-ab-comparison/task.md  
  tests/  
    selftest_manifest.md  
  reports/  
    final_report_template.md  
    ■■skill■■■■_T3-1-1■■■■.md
```

SKILL.md 是评测时主要加载的入口文件，保持短而强约束；references/ 存放按任务类型延迟读取的资料；scripts/ 提供仓库扫描、模式索引、自测任务生成和提交完整性检查；examples/ 记录 7 个自测任务（含 2 个从零开发算子和 1 个 skill 前后 A/B 对比）；tests/selftest_manifest.md 汇总自测状态。

3. 设计原则

本 skill 按以下原则设计：

1. 正确性优先：任何算子实现必须先和 PyTorch 或仓库已有可信实现对齐。
2. 最小补丁：避免大面积格式化、无关重构或修改编译器核心机制。
3. 仓库风格优先：实现前必须搜索并阅读相似 operator、test、benchmark、AOT 或 debugging 代码。
4. NineToothed 语义显式化：在写 application 前先设计 arrangement，确认各参数 arranged 后的外层 launch shape 对齐。
5. 布局不默认连续：涉及 stride、offset、non-contiguous 时必须写出假设、测试或不支持说明。
6. 性能结论必须有证据：没有 benchmark 命令、输入规模、dtype、设备和结果时，不允许声称优化成功。
7. 失败必须闭环：记录现象、命令、根因、最小修复或规避方案、复验结果。

4. 核心 workflow

SKILL.md 要求 AI 智能体按如下步骤执行：

1. 复述任务契约：算子语义、输入输出、shape、dtype、broadcast、边界、layout 和不支持范围。
2. 阅读仓库：用搜索定位相似算子、测试、benchmark、AOT、generated source 和 debugging 代码。
3. 设计 arrangement：写明每个 arranged tensor 的外层 launch shape 和 per-program block shape。
4. 实现最小补丁：优先使用仓库已有的 arrange-and-apply 或 @ninetoothed.jit 风格。
5. 添加 correctness test：对齐 PyTorch 或已有实现，覆盖普通 case 和边界/layout case。
6. 运行 targeted pytest：先跑最小相关测试，再扩大范围。
7. 对性能敏感任务补 benchmark：记录 baseline、规模、命令、结果和结论。
8. 对失败任务做闭环诊断：记录第一处关键错误、根因判断、修复或规避建议。
9. 输出审计摘要：列出修改文件、测试命令、benchmark、unsupported cases 和残余风险。

5. NineToothed 知识沉淀

本 skill 在 references/nine-toothed-api-notes.md 中沉淀了以下关键知识：

- Tensor 是 symbolic tensor，定义 kernel 时不保存真实数据，而保存 symbolic shape、stride、dtype 层级和 meta-operation 历史。
- NineToothed 的核心范式是 arrange-and-apply：先通过 tile、expand、squeeze、flatten、ravel、permute 等 meta-operation 建立 block 映射，再在 application 中定义每个 program 对 block 的计算。
- arranged 后的非标量参数外层 shape 应对齐，否则编译器无法按统一 grid 正确映射程序。
- reduction 或边界 tile 需要正确选择 Tensor(..., other=identity)，例如 softmax/max 类任务常用 float("-inf")。
- 复杂 arrangement 可以通过 ninetoothed.debugging.simulate_arrangement 或 concrete arranged.eval() 检查映射。

- AOT / generated source 任务需要明确区分 PyTorch CUDA runtime 可用和 CUDA Toolkit nvcc 可用，二者不是一回事。

6. 自测环境

自测分两轮，均为租借云 GPU：

第一轮（2026-07-01, runtime-only 镜像）：

```
GPU: NVIDIA GeForce RTX 4090 24GB
Driver: 570.169
CUDA Version reported by nvidia-smi: 12.8
OS image: Ubuntu 22.04, CUDA 12.8, PyTorch image
Python: 3.12.11
PyTorch: 2.9.1+cu128
Triton: 3.5.1
NineToothed: public repository snapshot (master c9ebd49), editable install
Known blocker: AOT tests require nvcc, which was not present in PATH
```

第二轮（2026-07-02, 完整 CUDA Toolkit 镜像, nvcc 12.8 / V12.8.61）：同款 RTX 4090、torch 2.9.1+cu128、Triton 3.5.1。第一轮的 AOT blocker 在本轮闭环复验（见第 11 节），并完成从零算子开发（第 12、13 节）、layout 修复复验（第 10 节）和 skill 前后 A/B 对比（第 14 节）。全套剩余测试 200 passed in 737.82s。

公开仓库由于远端访问 GitHub 超时，采用本地已拉取的 InfiniTensor/ninetoothed 和 InfiniTensor/ninetoothed-examples 快照上传到远端 GPU 实例后运行。

7. 自测任务总览

任务	类型	correctness 结果	benchmark / 诊断结果
01-elementwise-add	逐元素 / broadcast	examples add 1 passed in 5.58s	benchmark 1 passed in 9.80s ；自定义计时 NineToothed 0.0512 ms, PyTorch 0.0205 ms
02-softmax-reduction	归约 / block	core softmax 1 passed in 1.86s	examples benchmark 被 Triton exact-match precheck 阻塞；自定义计时 NineToothed 0.0543 ms, PyTorch 0.0215 ms
03-layout-stride-offset	布局敏感	根因定位 + 两种修复均 GPU 复验 allclose=True （闭环）	记录为布局诊断案例
04-performance-diagnosis	性能 / 诊断 / AOT	core add 和 softmax 通过	AOT blocker（缺 nvcc）已在 nvcc 镜像闭环复验： 11 passed, 1 skipped in 892.77s
05-from-scratch-gelu	逐元素（从零开发）	4/4 allclose vs torch GELU(tanh)，含非 2 幂和边界尺寸	0.99x – 1.05x vs PyTorch（2^16 – 2^24），无回退；含 NameError 诊断闭环
06-from-scratch-l2norm	归约（从零开发）	4/4 allclose, fp16/fp32, 奇数尺寸	融合单 kernel，比 PyTorch 快 20 – 28%（0.72x – 0.80x 用时）
07-ab-comparison	元评测：skill 前后 A/B	A 8/8；B 9/9（含 non-contiguous view）	行为清单 A 8.5/10, B 10/10；详见第 14 节

8. 自测任务一：逐元素 Add

任务目标是验证逐元素 add 类算子的 correctness 和 benchmark 流程。公开 examples 仓库中已有 ops.ninetoothed.torch.add，本次自测不修改源代码，而是验证 skill 是否能引导 AI 找到相似实现、运行正确性测试并记录性能结论。

正确性命令：

```
/usr/local/miniconda3/envs/py312/bin/python -m pytest tests/test_ops.py::TestAdd::test_correctness -q
```

结果：

```
1 passed in 5.58s
```

benchmark 命令：

```
/usr/local/miniconda3/envs/py312/bin/python -m pytest tests/test_benchmarks.py::TestAddBenchmark::test_benchmark -q -m
```

结果：

```
1 passed in 9.80s
```

额外自定义计时：

```
shape: (98432,)
dtype: torch.float16
device: RTX 4090
add allclose: True
NineToothed: 0.05119999870657921 ms
PyTorch: 0.020479999482631683 ms
```

结论：该逐元素 self-test 的正确性通过。小规模自定义计时中 PyTorch 更快，因此报告中不声称 NineToothed add 在该输入规模上更优，而强调 skill 会要求 AI 使用 benchmark 证据支撑性能判断。

9. 自测任务二：Softmax 归约

任务目标是验证 row-wise softmax 类归约算子的稳定公式、边界处理和 benchmark 诊断。核心仓库 tests/test_softmax.py 通过；examples 仓库的三方对比测试暴露出 Triton exact-match tolerance 过严导致的失败。

核心正确性命令：

```
/usr/local/miniconda3/envs/py312/bin/python -m pytest tests/test_softmax.py -q
```

结果：

```
1 passed in 1.86s
```

examples 三方对比命令：

```
/usr/local/miniconda3/envs/py312/bin/python -m pytest tests/test_ops.py::TestSoftmax::test_correctness -q
```

结果：

```
1 failed in 5.67s
AssertionError: NineToothed and Triton outputs differ.
```

benchmark 命令：

```
/usr/local/miniconda3/envs/py312/bin/python -m pytest tests/test_benchmarks.py::TestSoftmaxBenchmark::test_benchmark -q -m
```

结果：

```
1 failed in 7.85s
Failure occurred during benchmark correctness precheck because Triton exact-match tolerance was set to {"atol": 0, "rtol": 1e-05}
```

额外自定义计时：

```
shape: (4096, 781)
dtype: torch.float16
device: RTX 4090
softmax allclose_atol_1e-3: True
max_abs_diff_vs_torch: 1.52587890625e-05
NineToothed: 0.05432000011205673 ms
PyTorch: 0.021503999829292297 ms
```

结论：NineToothed softmax 在测试输入上与 PyTorch 在 fp16 容差内对齐。examples benchmark 的失败不是 PyTorch 参考不匹配，而是 Triton exact-match precheck 阻塞。该案例被纳入 skill 的 failure-diagnosis 流程，要求 AI 区分算子语义错误、参考实现误差和 benchmark harness 约束。

10. 自测任务三：Non-contiguous / Stride 布局

任务目标是验证 skill 是否会强制 AI 关注 non-contiguous、stride 和 offset。测试输入通过 stepped view 构造：

```
base_a = torch.randn(256, 512, device="cuda", dtype=torch.float16)
base_b = torch.randn(256, 512, device="cuda", dtype=torch.float16)
a = base_a[:, ::2]
b = base_b[:, ::2]
```

测试命令：

```
/usr/local/miniconda3/envs/py312/bin/python /root/ninetoothed-skill-work/selftest_custom.py
```

结果：

```
layout shape (256, 256) stride (512, 2) is_contiguous False
layout add allclose False
```

结论（第一轮）：公开 examples 的 add wrapper 对该 non-contiguous stepped view 未能与 PyTorch 精确对齐。

根因（第二轮闭环）：通过源码追踪定位到两个叠加原因——

1. examples 的 add 是 rank-1 kernel；把 2-D stepped view 传入时，launch 端静默只取 `size(0)/stride(0)`，实际只处理第 0 列，且不报任何错误（rank 不匹配时生成 kernel 的 stride-awareness 失效）。
2. wrapper 用 `torch.empty_like` 分配输出；对 non-dense view，`empty_like` 回退为 contiguous 布局，导致写入布局与读取布局不一致，其余位置为未初始化垃圾值。

修复与复验（`verify_layout_fix.py`，RTX 4090 实测）：

```
repro:  allclose=False, column-0-correct=True  ■■■■■■
fix (a): flatten guard■■■■■ reshape/contiguous ■■■■→ allclose=True
fix (b): rank-2 kernel■Tensor(2) + tile((1, BLOCK_SIZE))■stride-aware■→ allclose=True
```

结论：闭环完成。该案例沉淀为 skill 规则：kernel rank 必须与输入 rank 匹配（rank 不匹配会静默错算而非报错）；对 view 输入不要用 `empty_like` 分配输出。两条均已写入 `references/nine-toothed-api-notes.md` 的 Frequent Mistakes。

11. 自测任务四：性能 / AOT / 诊断

任务目标是验证 skill 在性能和构建失败场景中的诊断闭环。基础 runtime correctness 已通过：

```
tests/test_add.py: 1 passed in 8.69s
tests/test_softmax.py: 1 passed in 1.86s
```

AOT smoke test 初始失败包含两个阶段：

1. 远端环境 shell 中 python 不在 PATH，测试调用子进程时找不到 python。
2. 将 `/usr/local/miniconda3/envs/py312/bin` 加入 PATH 后，AOT 测试继续失败，关键错误为：

```
FileNotFoundError: [Errno 2] No such file or directory: 'nvcc'
```

根因判断：PyTorch CUDA runtime 和 NVIDIA driver 可用，并不代表 CUDA Toolkit 编译器可用。AOT 路径需要调用 nvcc 生成或编译代码，而当前租用镜像只保证 PyTorch CUDA runtime，未暴露 nvcc。

规避方案：

1. ■■■■■■ CUDA Toolkit ■■■■
2. ■■■/■■■ CUDA Toolkit■
3. ■■ PATH=/usr/local/cuda/bin:\$PATH■
4. ■ nvcc --version ■■■■■■ AOT tests■

该案例证明 skill 的诊断要求是必要的：AI 不能把 AOT 失败简单归结为算子代码错误，而要区分 runtime、toolchain、PATH、generated source 和 test harness 问题。

闭环复验（第二轮，nvcc 12.8 / V12.8.61 镜像）：

```
nvcc --version # Build cuda_12.8.r12.8/compiler.35404655_0
python -m pytest tests/test_aot.py -q
```

11 passed, 1 skipped in 892.77s (0:14:52)

auto-tuning 相关测试 4 passed in 147.50s；其余全套测试 200 passed in 737.82s

(0:12:17)。确认根因判断正确：失败是 toolchain/镜像供给问题，不是算子或编译器缺陷。附带经验：AOT 测试每个配置都调用 nvcc，12 个测试约 15 分钟，评测时间预算需考虑。

12. 自测任务五：从零开发 GELU（逐元素）

不参考任何现成 GELU 实现，按 skill workflow 从零开发 tanh 近似 GELU 算子。开发过程中出现一次真实编译失败并闭环：

```
NameError: name 'SQRT_2_OVER_PI' is not defined
```

根因：NineToothed 提取 application 源码时不携带模块级 Python 常量（提取后的源码独立编译），因此 application 内引用模块级常量会在 Triton 编译期抛 NameError。最小修复：把常量内联进 application 体。该 pitfall 已写入 references/nine-toothed-api-notes.md。

结果（RTX 4090）：

```
correctness: 4/4 allclose vs torch GELU(tanh)■■■ 2 ■■■■■■■■
benchmark:   0.99x-1.05x vs PyTorch■■■ 2^16-2^24■■■■■■■
```

13. 自测任务六：从零开发 row-wise L2-normalize（归约）

从零开发 $\text{output}[i,:] = \text{input}[i,:] / \max(\|\text{input}[i,:]\|_2, \text{eps})$ ，融合单 kernel：Tensor(2, other=0) 行 tile（0 是平方和的归约恒等元）、Tensor(0) 传标量 eps、fp32 累加。

结果（RTX 4090）：

```
correctness: 4/4 allclose vs PyTorch ■■■fp16/fp32■■■■■■■
benchmark:   0.72x-0.80x of PyTorch time — ■ PyTorch ■■■■■■ 20-28%
```

这是本提交中 NineToothed 实测超过 PyTorch 的案例：归约+缩放融合为单 kernel，省掉中间张量与多次 kernel launch。

14. 使用 skill 前后 A/B 对比实测

不同于第一版报告仅给出“预期对比”，本版完成了受控 A/B 实测（协议与完整记录见 examples/07-ab-comparison/task.md）：同一智能体、同一仓库快照（master c9ebd49）、同一 GPU，实现同一个新算子 softmax-temperature ($\text{output}[i,:] = \text{softmax}(\text{input}[i,:] / T)$ ，T 为运行时标量)。A 组不装 skill，B 组先读 SKILL.md。

GPU 实测：两组产物均全部通过（A 8/8, B 9/9 correctness；benchmark 均为 torch 非融合基线用时的 0.50x – 0.75x）。按预先定义的 10 项行为清单评分：A 8.5/10, B 10/10。

B 组独有行为（正是隐藏任务评分维度）：

- 编码前完整复述任务契约（dtype 规则、layout 规则、边界行为、不支持范围）；
- 补测 non-contiguous 转置 view（A 组完全没有测非连续输入）；
- 引用 api-notes 的 empty_like-on-views pitfall，选择 torch.empty(shape) 分配输出；
- 论证并否决 constexpr=True 传 T 的替代方案（避免按温度值重编译）；
- 输出残余风险清单及一行回退方案。

披露：A 组日志显示其读过工作区中已含 skill 经验的自测脚本（间接沾染），该偏差方向有利于 A 组，故实测差距是下界。

15. 使用 skill 前后对比（约束设计）

维度	不使用 skill 的常见风险	使用本 skill 的约束
任务理解	直接写代码，遗漏 dtype、broadcast、layout	必须先复述 shape、dtype、broadcast、layout、边界和不支持范围
仓库风格	新建不符合风格的 helper 或大改结构	必须先搜索相似 operator/test/benchmark/AOT 代码
arrangement	写 application 前未检查 block 映射	必须确认 arranged outer shape 和 per-program block shape
correctness	只跑粗粒度测试或不跑测试	必须运行 targeted pytest 并记录命令和结果
性能	口头声称优化	必须给出 benchmark 命令、输入规模、dtype、设备和结论
layout	默认 contiguous	必须测试或说明 non-contiguous、stride、offset 支持边界
失败诊断	只给模糊原因	必须记录现象、根因、最小修复或规避、复验结果

16. 适用范围与不适用范围

适用范围：

- 逐元素和 broadcast 类算子；
- reduction、softmax、pooling、normalization 等 block / reduction 类任务；
- non-contiguous、stride、offset 等布局敏感任务；
- correctness test、PyTorch reference 对齐；
- benchmark、generated source、AOT build、failing test 诊断；
- 最小补丁和仓库风格一致性检查。

不适用范围：

- 修改 NineToothed 编译器核心设计；
- 针对隐藏评测任务的硬编码答案；
- 依赖私有账号、密钥或在线服务才能复现的流程；
- 未安装 CUDA Toolkit 时强行完成 AOT 编译；
- 未经验证的所有 dtype、所有动态 shape、所有非连续布局。

17. 安全、合规与复现

本 skill 不包含 API key、账号凭据、隐藏评测答案或与赛题无关的个人信息。HONOR_CODE.md 中明确声明不绕过测试、不伪造结果、不硬编码隐藏任务。REFERENCE.md 中披露了公开仓库、官方文档、飞书 wiki、课程页面、PyTorch、Triton 和 AI 辅助范围。

提交包提供 scripts/check_submission.py 进行结构检查：

```
python scripts/check_submission.py --skill-dir .
```

当前检查结果：

Submission check passed

原始远端日志已保存在 outputs/remote-logs/（第一轮）和 gpu-session/logs/（第二轮，含 gelu/l2norm/layout_fix/aot/aot_autotune/pytest_rest/ab_run_a/ab_run_b 日志），报告中的测试结果均来自这些命令和日志文件。

18. 后续维护计划

后续维护重点包括：

1. 在正式比赛指定仓库中重新生成 repository-pattern-index；
2. 为更多 operator family 补充 examples，例如 rms_norm、max_pool2d、addmm；
3. 在最终提交前将小组名称、成员姓名、正式仓库 commit 补齐。

19. 结论

nine-toothed-operator-dev 将 NineToothed 算子开发中容易被 AI 忽略的关键动作固化为 .skill workflow，覆盖算子语义抽取、仓库检索、arrangement 设计、correctness test、benchmark、layout 诊断、AOT/generated source 诊断和最终审计摘要。

本版报告的四个证据闭环均已在 GPU 上实测完成：

1. 两个从零开发算子（GELU、L2-normalize）correctness 全通过并带 benchmark，其中 L2-normalize 比 PyTorch 快 20 – 28%，GELU 开发过程含一次真实 NameError 诊断闭环；
2. non-contiguous add 失败案例完成"复现 → 根因（rank 静默不匹配 + empty_like 回退）→ 两种修复 → GPU 复验"全闭环；
3. AOT blocker 在含 nvcc 的镜像上闭环复验：tests/test_aot.py 11 passed 1 skipped，全套其余测试 200 passed；
4. skill 前后 A/B 对比实测：B（带 skill）在 10 项行为清单上 10/10，多出契约复述、non-contiguous 覆盖、pitfall 引用决策和残余风险文档等隐藏任务评分直接相关的行为。

skill 的 references 亦已逐条对照仓库（master c9ebd49）校验修正。提交材料齐备，只待按赛题组指定的提交仓库格式放入规定目录并发起 PR。